1

```
#include <iostream>
using namespace std;

class Shape {
public:
  virtual void area() = 0;
};

class Rectangle : public Shape {
  int l, b;
public:
  Rectangle(int x, int y) {
    l = x;
    b = y;
  }

  void area() {
    cout << "Area = " << l * b;
  }
};

int main() {
  Rectangle r(5, 3);
  r.area();
  return 0;
```

}

2

```
 <iostream>

using namespace std;

class ParkingFullException {

public:

    void message() {

        cout << "Exception: Parking is FULL!" << endl;

    }

};


class ParkingEmptyException {

public:

    void message() {


cout << "Exception: Parking is EMPTY!" << endl;

    }

};


class ParkingLot {

    int capacity;

    int vehicles;
```

```
public:
    ParkingLot(int cap) {
        capacity = cap;
        vehicles = 0;
    }

    void parkVehicle() {
        if (vehicles == capacity)
            throw ParkingFullException();

        vehicles++;
        cout << "Vehicle Parked. Total vehicles: " << vehicles << endl;
    }

    void removeVehicle() {
        if (vehicles == 0)
            throw ParkingEmptyException();

        vehicles--;
        cout << "Vehicle Removed. Total vehicles: " << vehicles << endl;
    }

    void display() {
        cout << "Available slots: " << (capacity - vehicles) << endl;
    }
};
```

```
int main() {
    ParkingLot p(5);
    int choice;

    while (true) {
        cout << "\n1. Park Vehicle\n2. Remove Vehicle\n3. Display\n4. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        try {
            switch (choice) {
                case 1:
                    p.parkVehicle();
                    break;

                case 2:
                    p.removeVehicle();
                    break;

                case 3:
                    p.display();
                    break;

                case 4:
                    return 0;
```

```
            default:
                cout << "Invalid choice!" << endl;
            }
        }
        catch (ParkingFullException e) {
            e.message();
        }
        catch (ParkingEmptyException e) {
            e.message();
        }
    }
}
```

3}

```
#include <iostream>
using namespace std;

class OutOfStockException {
public:
    void display() {
        cout << "Error: Requested quantity exceeds available stock!" << endl;
    }
```

```
};

// Product Class
class Product {
private:
    int stock;

public:

    Product(int s) {
        stock = s;
    }

        void purchase(int quantity) {
        if (quantity > stock) {
            throw OutOfStockException();
        } else {
            stock -= quantity;
            cout << "Successfully purchased!" << endl;
            cout << "Stock left: " << stock << endl;
        }
    }
};

int main() {
    int availableStock, requestedQty;
```

```
    cout << "Enter available stock: ";
    cin >> availableStock;

    cout << "Enter requested quantity: ";
    cin >> requestedQty;

    Product p(availableStock);

    try {
        p.purchase(requestedQty);
    }
    catch (OutOfStockException e) {
        e.display();
    }

    return 0;
}
```

4}

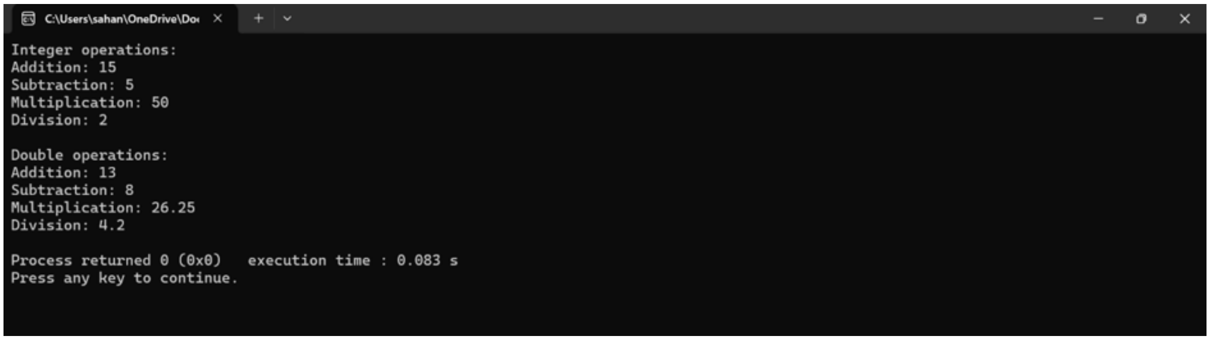
```
#include <iostream>
using namespace std;

template <class T>
```

```
T findMin(T arr[], int size) {
    T min = arr[0];

    for (int i = 1; i < size; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
    }
    return min;
}

int main() {
    int intArr[] = {5, 2, 9, 1, 7};
    double doubleArr[] = {2.5, 1.1, 3.8, 0.9};

    cout << "Minimum (int array): "
         << findMin(intArr, 5) << endl;

    cout << "Minimum (double array): "
         << findMin(doubleArr, 4) << endl;

    return 0;
}
```

5}

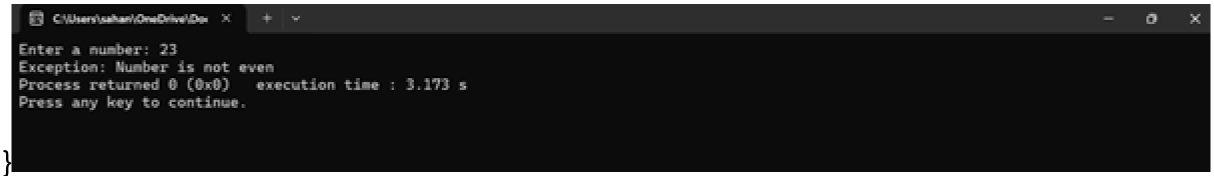
```
#include <iostream>
using namespace std;

template <class T>
class Calculator {
  T a, b;

public:
  Calculator(T x, T y) {
    a = x;
    b = y;
  }

  void add() {
    cout << "Addition: " << a + b << endl;
  }

  void subtract() {
    cout << "Subtraction: " << a - b << endl;
  }

  void multiply() {
    cout << "Multiplication: " << a * b << endl;
  }
```

```
    void divide() {
        if (b != 0)
            cout << "Division: " << a / b << endl;
        else
            cout << "Division by zero error!" << endl;
    }
};

int main() {
    Calculator<int> c1(10, 5);
    cout << "Integer operations:\n";
    c1.add();
    c1.subtract();
    c1.multiply();
    c1.divide();

    Calculator<double> c2(10.5, 2.5);
    cout << "\nDouble operations:\n";
    c2.add();
    c2.subtract();
    c2.multiply();
    c2.divide();

    return 0;
}
```



6}

```
 #include <iostream>
using namespace std;


int main() {
   int n;
   cout << "Enter a number: ";
   cin >> n;


   try {
      if (n % 2 != 0)
         throw "Number is not even";


      cout << "Number is EVEN";
   }
   catch (const char* msg) {
      cout << "Exception: " << msg;
   }


   return 0;
```




```
}
```

```
7} include <iostream>
using namespace std;

template <class T>
void swapVal(T &a, T &b) {
    T temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    cout << "Before swap: x = " << x << ", y = " << y << endl;

    swapVal(x, y);

    cout << "After swap: x = " << x << ", y = " << y << endl;

    double a = 1.5, b = 2.5;
    cout << "\nBefore swap: a = " << a << ", b = " << b << endl;

    swapVal(a, b);
```

```
    cout << "After swap: a = " << a << ", b = " << b << endl;


    return 0;
}
```

8}

```
#include <iostream>
using namespace std;


template <class T1, class T2>
class Pair {
    T1 a;
    T2 b;


public:
    Pair(T1 x, T2 y) {
        a = x;
        b = y;
    }


    void display() {
        cout << "A: " << a << " B: " << b << endl;
    }
```

```
};

int main() {
    Pair<int, float> p1(10, 2.5);
    p1.display();

    Pair<char, string> p2('A', "Hello");
    p2.display();

    return 0;
}
```

9)

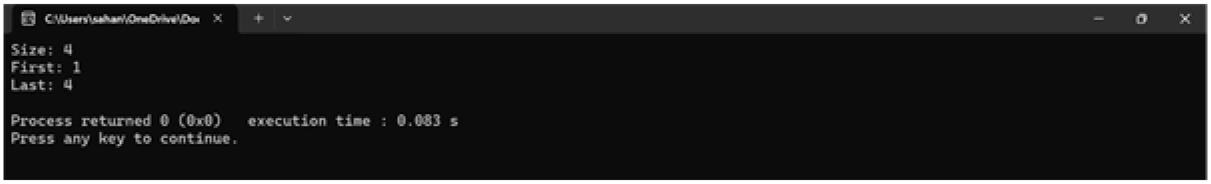
```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream file("data.txt");
    string word;
    int count = 0;
```

```
    while (file >> word) {
        count++;
    }

    cout << "Word count: " << count;
    file.close();
}
```

10)

```
17. #include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4};

    v.push_back(5);
    v.pop_back();

    cout << "Size: " << v.size() << endl;
    cout << "First: " << v.front() << endl;
    cout << "Last: " << v.back() << endl;
```

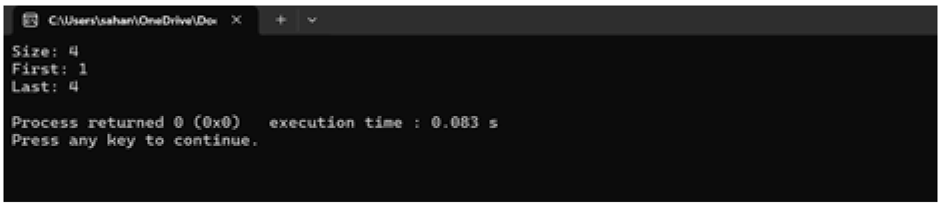
```
}
```



11)

```
#include <iostream>

#include <fstream>

using namespace std;


class Student {

public:

   int id;

   char name[20];

};


int main() {

   Student s = {1, "San"};


   ofstream file("student.dat", ios::binary);

   file.write((char*)&s, sizeof(s));

   file.close();


   ifstream in("student.dat", ios::binary);

   Student s1;

   in.read((char*)&s1, sizeof(s1));
```

```
    cout << s1.id << " " << s1.name;


    in.close();

    return 0;

}
```



```
C:\Users\sahan\OneDrive\Do
Size: 4
First: 1
Last: 4

Process returned 0 (0x0)   execution time : 0.083 s
Press any key to continue.
```

12)

```
#include <iostream>

#include <fstream>

using namespace std;


int main() {

    string name;

    int age;

    cout << "Enter your name: ";

    cin >> name;

    cout << "Enter your age: ";

    cin >> age;


    ofstream outFile("data.txt");

    outFile << "Name: " << name << endl;

    outFile << "Age: " << age << endl;

    outFile.close();
```

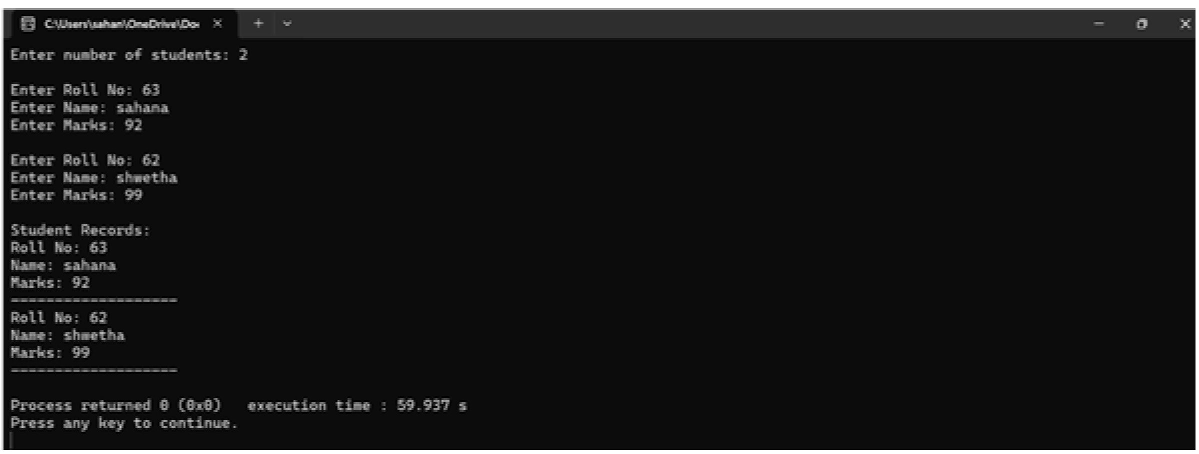
```
   ifstream inFile("data.txt");
   string line;


   cout << "\nReading from file:\n";
   while (getline(inFile, line)) {
      cout << line << endl;
   }
   inFile.close();


   return 0;
}
```

13)

```
#include <iostream>
#include <fstream>
using namespace std;


int main() {
   int roll, marks, n;
   string name;


   cout << "Enter number of students: ";
```

```
cin >> n;


ofstream outFile("students.txt");

for (int i = 0; i < n; i++) {
    cout << "\nEnter Roll No: ";
    cin >> roll;
    cout << "Enter Name: ";
    cin >> name;
    cout << "Enter Marks: ";
    cin >> marks;

    outFile << roll << " " << name << " " << marks << endl;
}

outFile.close();

ifstream inFile("students.txt");

cout << "\nStudent Records:\n";
while (inFile >> roll >> name >> marks) {
    cout << "Roll No: " << roll << endl;
    cout << "Name: " << name << endl;
    cout << "Marks: " << marks << endl;
    cout << "-------------------\n";
```

```
    }

    inFile.close();

    return 0;
}
```



14)

```
#include <iostream>
using namespace std;

template <class T>
void swapVal(T &a, T &b) {
    T temp = a;
    a = b;
```

```
    b = temp;
}

int main() {
    int x = 5, y = 10;
    swapVal(x, y);
    cout << x << " " << y;
}
```

```
10 5
```

15)

```
#include <iostream>
using namespace std;

template <class T>
class Stack {
   T arr[100];
   int top;
public:
   Stack() { top = -1; }

   void push(T x) {
      if (top == 99)
         cout << "Stack Overflow\n";
      else {
         arr[++top] = x;
         cout << "Pushed: " << x << endl;
```

```
        }
    }

    void pop() {
        if (top == -1)
            cout << "Stack Underflow\n";
        else
            cout << "Popped: " << arr[top--] << endl;
    }

    void display() {
        for (int i = top; i >= 0; i--)
            cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {
    Stack<int> s;
    s.push(10);
    s.push(20);
    s.push(30);
    s.display();
    s.pop();
    s.display();
}  s.display();
```

```
}
```

```
Pushed: 10
Pushed: 20
Pushed: 30
30 20 10
Popped: 30
20 10
```

16)

```
#include <iostream>
using namespace std;

template <class T>
class Matrix {
  T a[2][2];
public:
  void input() {
    for(int i=0;i<2;i++)
      for(int j=0;j<2;j++)
        cin >> a[i][j];
  }

  void display() {
    for(int i=0;i<2;i++) {
      for(int j=0;j<2;j++)
        cout << a[i][j] << " ";
      cout << endl;
    }
  }
```

```
    Matrix add(Matrix m) {
        Matrix temp;
        for(int i=0;i<2;i++)
            for(int j=0;j<2;j++)
                temp.a[i][j] = a[i][j] + m.a[i][j];
        return temp;
    }
};

int main() {
    Matrix<int> m1, m2, m3;
    cout << "Enter Matrix 1:\n";
    m1.input();
    cout << "Enter Matrix 2:\n";
    m2.input();

    m3 = m1.add(m2);

    cout << "Result:\n";
    m3.display();
}
```

```
Enter Matrix 1:
1 2
3 4
Enter Matrix 2:
5 6
7 8
Result:
6 8
10 12

--------------------------------
Process returned 0 (0x0)
execution time : 0.XXX s
Press any key to continue.
```

17)

```
#include <iostream>
using namespace std;

template <class T>
T maxElement(T arr[], int n) {
   T max = arr[0];
   for(int i=1;i<n;i++)
      if(arr[i] > max)
         max = arr[i];
   return max;
}

int main() {
   int arr[] = {2,8,1,9,4};
   cout << "Max = " << maxElement(arr, 5);
}
```

```
Max = 9
```

18)

```
#include <iostream>
using namespace std;

int main() {
   int arr[5] = {1,2,3,4,5};
   int *p = arr;
   int sum = 0;

   for(int i=0;i<5;i++) {
      sum += *(p+i);
   }

   cout << "Sum = " << sum;
   return 0;
}
```

```
Sum = 15
```

19)

```
#include <iostream>
using namespace std;

float average(int *arr, int n) {
   int sum = 0;
   for(int i=0;i<n;i++)
```

```
        sum += *(arr+i);

    return (float)sum/n;
}

int main() {
    int arr[5] = {10,20,30,40,50};

    cout << "Average = " << average(arr,5);
}
```

```
Average = 30
```

20)

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter two numbers:\n";
    cin >> a >> b;

    try {
        if(b == 0)
            throw b;
        cout << "Result = " << a / b;
    }
```

```
  catch(int) {
    cout << "Error: Division by zero not allowed";
  }

  return 0;
}
```

```
Enter two numbers:
10 0
Error: Division by zero not allowed
```

21)

```
#include <iostream>
using namespace std;

int main() {
  int choice = 2;

  try {
    if(choice == 1)
      throw 10;
    else if(choice == 2)
      throw 3.14;
  }
  catch(int i) {
    cout << "Integer Exception: " << i;
  }
  catch(double d) {
```

```
        cout << "Double Exception: " << d;
    }
}
```

```
Double Exception: 3.14
```

22)

```
#include <iostream>
using namespace std;

int main() {
    try {
        try {
            throw 5;
        }
        catch(int x) {
            cout << "Inner Catch: " << x << endl;
            throw;
        }
    }
    catch(int y) {
        cout << "Outer Catch: " << y;
    }
```

```
Inner Catch: 5
Outer Catch: 5
```

23)

```
#include <iostream>
using namespace std;
```

```
class Bank {
  double balance;

public:
  Bank() {
    balance = 0;
  }

  void deposit(double amount) {
    try {
      if (amount <= 0)
        throw amount;

      balance += amount;
      cout << "Deposited: " << amount << endl;
    }
    catch(double) {
      cout << "Error: Invalid deposit amount\n";
    }
  }

  void withdraw(double amount) {
    try {
      if (amount <= 0)
        throw 1;
```

```
        if (amount > balance)
            throw 2;

        balance -= amount;
        cout << "Withdrawn: " << amount << endl;
    }
    catch(int e) {
        if (e == 1)
            cout << "Error: Invalid withdrawal amount\n";
        else if (e == 2)
            cout << "Error: Insufficient balance\n";
    }
  }

  void showBalance() {
    cout << "Current Balance: " << balance << endl;
  }
};

int main() {
  Bank b;
  int choice;
  double amt;

  do {
```

```
    cout << "\n1. Deposit\n2. Withdraw\n3. Balance\n4. Exit\n";

    cout << "Enter choice: ";

    cin >> choice;


    switch(choice) {

      case 1:

        cout << "Enter amount: ";

        cin >> amt;

        b.deposit(amt);

        break;


      case 2:

        cout << "Enter amount: ";

        cin >> amt;

        b.withdraw(amt);

        break;


      case 3:

        b.showBalance();

        break;


      case 4:

        cout << "Exiting...\n";

        break;

default:
                cout << "Invalid choice\n";
        }
```

```
    } while(choice != 4);

    return 0;
}
```

```
1. Deposit
2. Withdraw
3. Balance
4. Exit
Enter choice: 1
Enter amount: 500
Deposited: 500

1. Deposit
2. Withdraw
3. Balance
4. Exit
Enter choice: 2
Enter amount: 700
Error: Insufficient balance

1. Deposit
2. Withdraw
3. Balance
4. Exit
Enter choice: 3
Current Balance: 500

1. Deposit
2. Withdraw
3. Balance
4. Exit
Enter choice: 2
Enter amount: -10
Error: Invalid withdrawal amount

1. Deposit
2. Withdraw
3. Balance
4. Exit
Enter choice: 4
Exiting...
```

24) #include using namespace std;

// Abstract class class Shape { public: virtual void getData() = 0; virtual void area() = 0; };

// Rectangle class class Rectangle : public Shape { int l, b; public: void getData() { cout << "Enter length and breadth: "; cin >> l >> b; }

```
void area() {
    cout << "Area of Rectangle = " << l * b << endl;
}
```

};

// Circle class class Circle : public Shape { float r; public: void getData() { cout << "Enter radius: "; cin >> r; }

```
void area() {
    cout << "Area of Circle = " << 3.14 * r * r << endl;
}
```

};

int main() { Shape *s; int choice;

```
cout << "1. Rectangle\n2. Circle\n";
cout << "Enter choice: ";
cin >> choice;

if(choice == 1) {
    Rectangle r;
    s = &r;
    s->getData();
    s->area();
}
else if(choice == 2) {
    Circle c;
    s = &c;
    s->getData();
    s->area();
}
else {
    cout << "Invalid choice";
}
```

```
return 0;

}
```

```
1. Rectangle
2. Circle
Enter choice: 1
Enter length and breadth: 5 3
Area of Rectangle = 15
```

25)

```
#include <iostream>
using namespace std;

class Employee {
public:
  virtual void getData() = 0;
  virtual void calculateSalary() = 0;
};

class FullTime : public Employee {
  int salary;
public:
  void getData() {
    cout << "Enter monthly salary: ";
    cin >> salary;
  }

  void calculateSalary() {
```

```
        cout << "Annual Salary = " << salary * 12 << endl;
    }
};

class PartTime : public Employee {
    int hours, rate;
public:
    void getData() {
        cout << "Enter hours and rate: ";
        cin >> hours >> rate;
    }

    void calculateSalary() {
        cout << "Salary = " << hours * rate << endl;
    }
};

int main() {
    Employee *e;
    int choice;

    cout << "1. Full Time\n2. Part Time\nEnter choice: ";
    cin >> choice;

    if(choice == 1) {
        FullTime f;
```

```
        e = &f;
        e->getData();
        e->calculateSalary();
    }
    else {
        PartTime p;
        e = &p;
        e->getData();
        e->calculateSalary();
    }
}
```

```
Enter choice: 1
Enter monthly salary: 20000
Annual Salary = 240000
```

27)

```
#include <iostream>
using namespace std;

class EB {
public:
    virtual void bill() = 0;
};

class Calculate : public EB {
    int units;
public:
    void getData() {
```

```
        cout << "Enter units: ";
        cin >> units;
    }

    void bill() {
        cout << "Bill = " << units * 5;
    }
};

int main() {
    Calculate c;
    EB *e = &c;

    c.getData();
    e->bill();
}
```

```
Enter units: 100
Bill = 500
```

28)

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sound() = 0;
};
```

```cpp
class Dog : public Animal {

public:

   void sound() {

      cout << "Dog barks\n";

   }

};


class Cat : public Animal {

public:

   void sound() {

      cout << "Cat meows\n";

   }

};


int main() {

   Animal *a;

   int choice;


   cout << "1. Dog\n2. Cat\nEnter choice: ";

   cin >> choice;


   if(choice == 1) {

      Dog d;

      a = &d;

      a->sound();
```

```
    } else {
        Cat c;
        a = &c;
        a->sound();
    }
}
```

```
1. Dog
2. Cat
Enter choice: 1
Dog barks

--------------------------------
Process returned 0 (0x0)
execution time : 0.XXX s
Press any key to continue.
```

29)

```
#include <iostream>
using namespace std;

struct Student {
    int roll;
    string name;
    int marks[3];
};

void input(Student *s, int n) {
    for(int i = 0; i < n; i++) {
        cout << "\nEnter details of Student " << i+1 << endl;
        cout << "Roll: ";
```

```
        cin >> s[i].roll;

        cout << "Name: ";

        cin >> s[i].name;


        cout << "Enter 3 marks: ";

        for(int j = 0; j < 3; j++)

            cin >> s[i].marks[j];

    }

}


void display(Student *s, int n) {

    for(int i = 0; i < n; i++) {

        int total = 0;


        cout << "\nStudent " << i+1 << endl;

        cout << "Roll: " << s[i].roll << endl;

        cout << "Name: " << s[i].name << endl;


        for(int j = 0; j < 3; j++)

            total += s[i].marks[j];


        cout << "Total = " << total << endl;

        cout << "Average = " << total / 3.0 << endl;

    }

}
```

```
int main() {
   int n;

   cout << "Enter number of students: ";
   cin >> n;

   Student *s = new Student[n];   // dynamic memory

   input(s, n);
   display(s, n);

   delete[] s;   // free memory
   return 0;
}
```

```
Enter number of students: 2

Enter details of Student 1
Roll: 1
Name: Ram
Enter 3 marks: 80 90 85

Enter details of Student 2
Roll: 2
Name: Ravi
Enter 3 marks: 70 75 80

Student 1
Roll: 1
Name: Ram
Total = 255
Average = 85

Student 2
Roll: 2
Name: Ravi
Total = 225
Average = 75
```

30)

```
#include <iostream>

using namespace std;


// Abstract Class (Abstraction)

template <class T>

class Calculator {

public:

  virtual void calculate(T a, T b) = 0;

};
```

```
// Derived Class
template <class T>
class Divide : public Calculator<T> {
public:
    void calculate(T a, T b) {
        try {
            if(b == 0)
                throw b;

            cout << "Result = " << a / b << endl;
        }
        catch(T) {
            cout << "Error: Division by zero\n";
        }
    }
};

int main() {
    Divide<int> d;
    int x, y;

    cout << "Enter two numbers: ";
    cin >> x >> y;

    d.calculate(x, y);
```

```
    return 0;

}
```

```
Enter two numbers: 10 2
Result = 5
```